



# Container Security



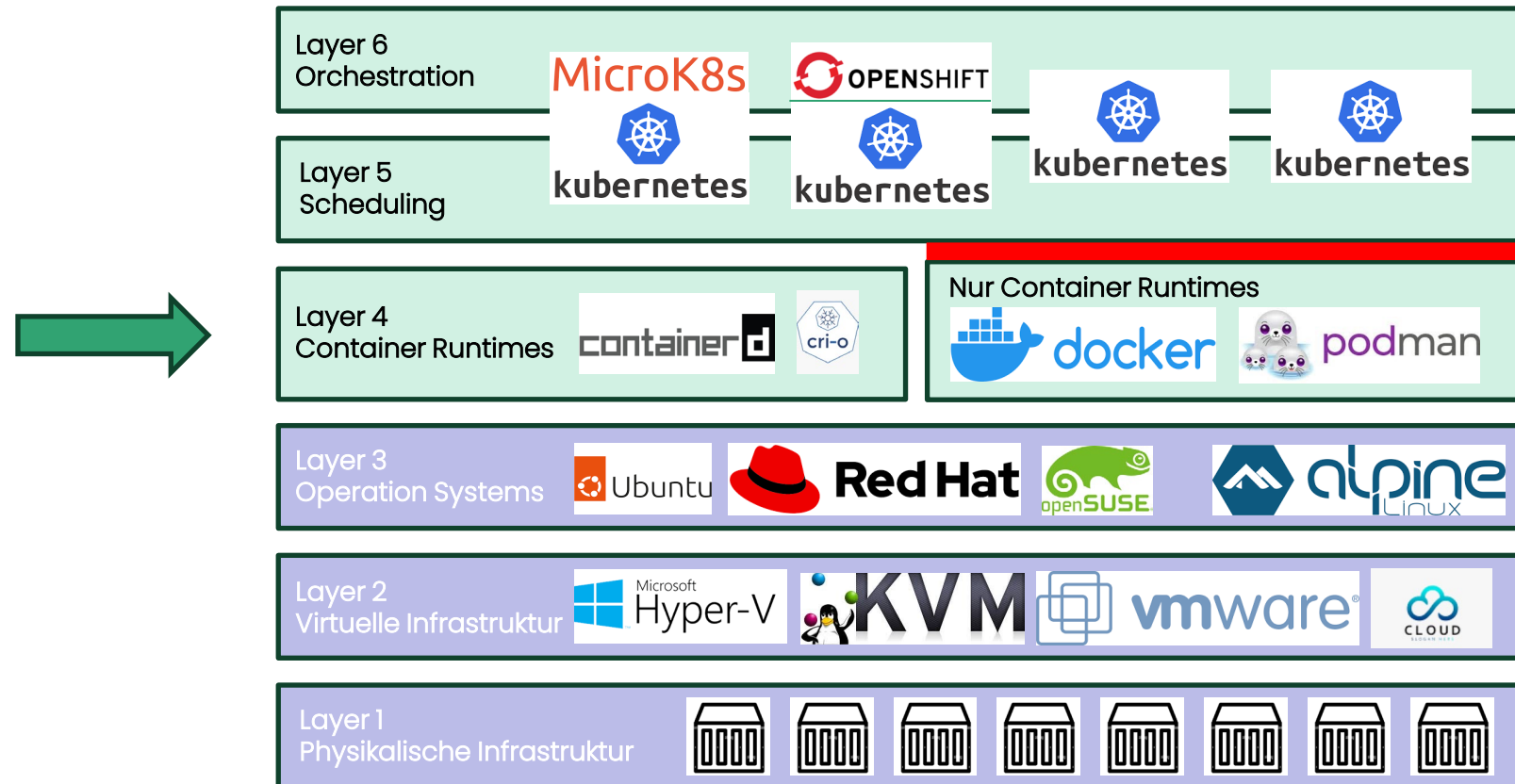
# Lernziele

- Sie haben einen Überblick welche Sicherheits-Aspekte bei Containern zu beachten sind.

# Zeitlicher Ablauf

- Limitierte Container-Instanzen
- Signierte Images
- Informationen im Internet
- Vulnerability Scanner
- Weitere
- Reflexion
- Lernzielkontrolle

# Container Runtimes und Kubernetes



Die (vereinfachten) »Layer« der Container-Welt

# Container-Instanzen Limitieren

- Die zu erzeugende/zu startende/gestartete Container-Instanz kann auch direkt limitiert werden.
- Dazu brauchen wir zunächst einen Überblick über die aktuell verbrauchten Ressourcen.
  - ◇ `docker [container] stats`
- Siehe Übung [03-3-Docker](#) und für Kubernetes: <http://blog.kubecost.com/blog/requests-and-limits/>
- Mögliche Limitierungen (**K8s auf Namespaces möglich**)
  - ◇ `--cpu-shares=512` – CPU shares (relative Gewichtung)
  - ◇ `--cpuset-cpus="2-3"` – [CPUs Pinning](#) (z. B.: 0-3, oder 0,1)
  - ◇ `--cpuset-mems="2,4"` – Speicherknoten (Memory Nodes – MEMs), in denen die Ausführung erlaubt ist, z. B.: 0-3 oder 0,1. Achtung: nur für NUMA-Systeme (eine Computer-Speicher-Architektur für Multiprozessorsysteme)
  - ◇ `--cpu-quota=25 #(% )` – Limitierung der CPU Quota via CFS (Completely Fair Scheduler): Limitiert so den maximalen CPU-Verbrauch des Containers
  - ◇ `--memory=""` – Maximal zulässiger Speicherverbrauch der Containers (Format: <number>[<unit>]). Number ist dabei ein positiver Integer-Wert, Units wie üblich: **b**, **k**, **m** oder **g**. Minimum sind **4M**.



# Übung 05-1: Container-Instanzen Limitieren

- Klickt auf den Link unten und spielt die Übung durch:

## Übung: Memory Limits

Das Begrenzen des Arbeitsspeichers (Memory) eines Docker-Containers ist ein wichtiger Aspekt der Ressourcenverwaltung und hilft sicherzustellen, dass einzelne Container die Systemressourcen nicht überbeanspruchen.

Zuerst starten wir einen Container ohne Begrenzung des Speichers

```
[17]: ! docker run --rm --name webshop-mem -d registry.gitlab.com/ch-mc-b/autoshop/shop:2.0.0
! docker stats --no-stream webshop-mem
```

| CONTAINER ID                                                     | NAME        | CPU %  | MEM USAGE / LIMIT   | MEM % | NET I/O   | BLOCK I/O   | PIDS |
|------------------------------------------------------------------|-------------|--------|---------------------|-------|-----------|-------------|------|
| c3ef74cd7f58d36a4862a1d03d834b9174bbeb83dd3a20592f972db62e6d7642 |             |        |                     |       |           |             |      |
| c3ef74cd7f58                                                     | webshop-mem | 25.58% | 24.47MiB / 7.755GiB | 0.31% | 606B / 0B | 77.8kB / 0B | 1    |

Dann starten wir den Container nochmals mit Begrenzung des Speichers:

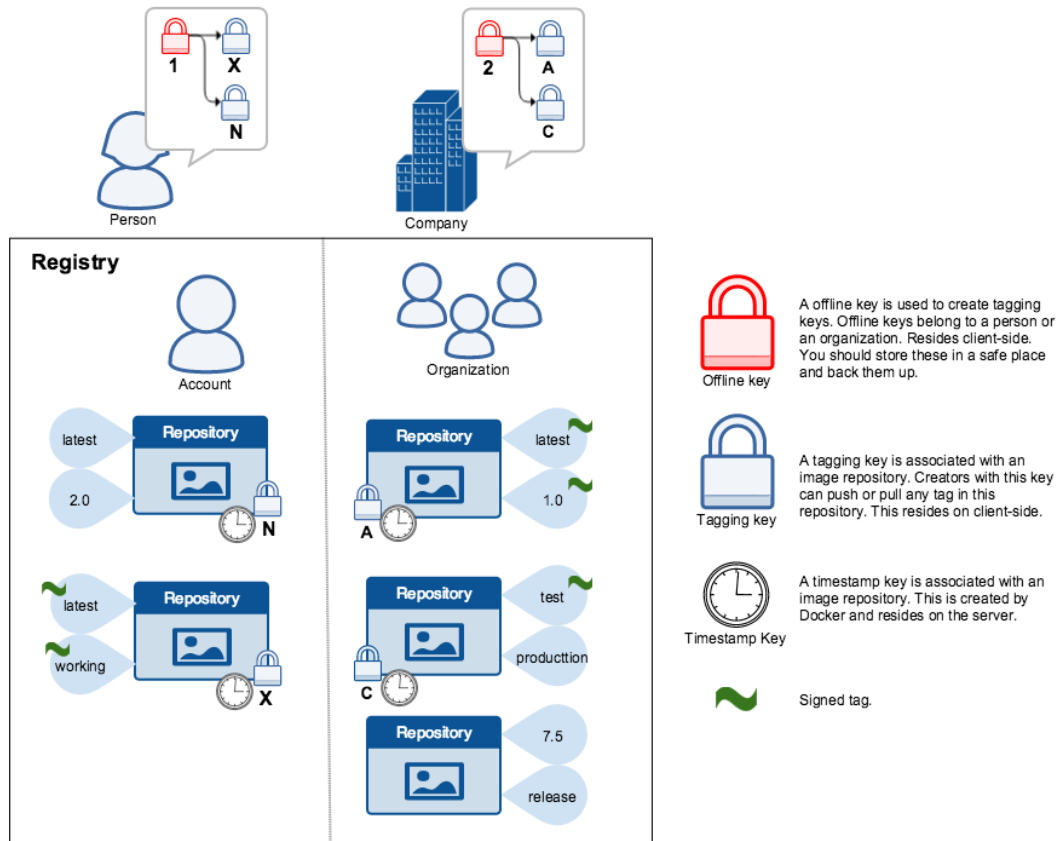
```
[18]: ! docker run --rm --name webshop-mem200 --memory=200m -d registry.gitlab.com/ch-mc-b/autoshop/shop:2.0.0
! docker stats --no-stream webshop-mem200
```

| CONTAINER ID                                                     | NAME           | CPU %  | MEM USAGE / LIMIT | MEM %  | NET I/O   | BLOCK I/O | PIDS |
|------------------------------------------------------------------|----------------|--------|-------------------|--------|-----------|-----------|------|
| 7a95453c87ae22764fe18774003ee8a35deb3e1b60bb3e4705c0aa9da582f65b |                |        |                   |        |           |           |      |
| 7a95453c87ae                                                     | webshop-mem200 | 25.39% | 24.41MiB / 200MiB | 12.20% | 306B / 0B | 0B / 0B   | 1    |

- Ergebnis:** Jeder Container läuft als eigener Prozess im Host-System und kann über **Ressourcenlimits** (z. B. CPU, RAM) gezielt eingeschränkt werden. Docker nutzt dafür **cgroups (Control Groups)** des Linux-Kernels.

**Erkenntnisse:** Durch cgroups lassen sich Container **kontrolliert und fair** Ressourcen teilen. Limits verhindern, dass einzelne Container **das System überlasten**. Damit wird eine **stabile, vorhersagbare Performance** in Multi-Container-Umgebungen erreicht.

# Signierte Images (Trusted Images)



- Buch Skalierbare Container-Infrastrukturen, Kapitel 4.7 mit SUSE/SLES mit sle2docker.

- Docker Handbuch

◇ Use Trusted Images:

<https://docs.docker.com/engine/security/trust/>

```
export DOCKER_CONTENT_TRUST=1
```

```
$ docker push <username>/trusttest:testing
The push refers to a repository [docker.io/<username>/trusttest] (len: 1)
9a61b6b1315e: Image already exists
902b87aaaec9: Image already exists
latest: digest: sha256:d02adacee0ac7a5be140adb94fa1dae64f4e71a68696e7f8e7cbf9db8dd49418 size: 3220
Signing and pushing trust metadata
You are about to create a new root signing key passphrase. This passphrase
will be used to protect the most sensitive key in your signing system. Please
choose a long, complex passphrase and be careful to keep the password and the
key file itself secure and backed up. It is highly recommended that you use a
password manager to generate the passphrase and keep it safe. There will be no
way to recover this key. You can find the key in your config directory.
Enter passphrase for new root key with id a1d96fb:
Repeat passphrase for new root key with id a1d96fb:
Enter passphrase for new repository key with id docker.io/<username>/trusttest (3a932f1):
Repeat passphrase for new repository key with id docker.io/<username>/trusttest (3a932f1):
Finished initializing "docker.io/<username>/trusttest"
```

# Docker Security:



## The Update Framework

A framework for securing software update systems

The Update Framework is a  
CNCF graduated project



- Vorbetrachtungen
  - ◇ Mangelhafte Sicherheit im Vergleich zu realen Maschinen oder VMs, was die Isolation der Namespaces angeht.
  - ◇ Keine Transparenz, was den Image Build angeht, sofern die Images aus externen Quellen stammen.
  - ◇ Vertrauenswürdigkeit von externen Quellen/Repos/Registries
- Per Default aktiv: TUF (The Update Framework)
  - ◇ TUF stellt keine konkrete Software oder ein Tool dar, sondern versteht sich auch als Spezifikation.
  - ◇ **TUF** kann im einfachsten Fall als Helfer für die Softwareverwaltung verstanden werden, welche z. B. auf Basis von digitalen Signaturen, Meta-Informationen und korrespondierenden Schlüsseln prüft, ob eine neuere Version einer Software verfügbar ist und diese auch als valide bzw. »**sauber**« anzusehen



# Microservice Patterns: The Twelve Factors App - <https://12factor.net/> (veraltet?)

- I. Codebase
  - ◇ One codebase tracked in revision control, many deploys
- II. Dependencies
  - ◇ Explicitly declare and isolate dependencies
- III. Config
  - ◇ Store config in the environment
- IV. Backing services
  - ◇ Treat backing services as attached resources
- V. Build, release, run
  - ◇ Strictly separate build and run stages
- VI. Processes
  - ◇ Execute the app as one or more stateless processes
- VII. Port binding
  - ◇ Export services via port binding
- VIII. Concurrency
  - ◇ Scale out via the process model
- IX. Disposability
  - ◇ Maximize robustness with fast startup and graceful shutdown
- X. Dev/prod parity
  - ◇ Keep development, staging, and production as similar as possible
- XI. Logs
  - ◇ Treat logs as event streams
- XII. Admin processes
  - ◇ Run admin/management tasks as one-off processes

# Aktuellere Seiten

**MITRE | ATT&CK®** Matrices ▾ Tactics ▾ Techniques ▾ Defenses ▾ CTI ▾ Resources ▾  
Benefactors Blog ↗ Search 🔍

ATT&CKcon 5.0 returns October 22-23, 2024 in McLean, VA. Stay tuned for registration details!

## ATT&CK®

[Get Started](#)
[Take a Tour](#)
[Contribute](#)
[Blog ↗](#)
[FAQ](#)
[Random Page ▾](#)

MITRE ATT&CK® is a globally-accessible knowledge base of adversary tactics and techniques based on real-world observations. The ATT&CK knowledge base is used as a foundation for the development of specific threat models and methodologies in the private sector, in government, and in the cybersecurity product and service community.

With the creation of ATT&CK, MITRE is fulfilling its mission to solve problems for a safer world — by bringing communities together to develop more effective cybersecurity. ATT&CK is open and available to any person or organization for use at no charge.

## ATT&CK Matrix for Enterprise

layout: side ▾

show sub-techniques

hide sub-techniques

| Reconnaissance<br>10 techniques        | Resource Development<br>8 techniques | Initial Access<br>10 techniques   | Execution<br>14 techniques             | Persistence<br>20 techniques           | Privilege Escalation<br>14 techniques | Defense Evasion<br>43 techniques      |
|----------------------------------------|--------------------------------------|-----------------------------------|----------------------------------------|----------------------------------------|---------------------------------------|---------------------------------------|
| Active Scanning (3)                    | Acquire Access                       | Content Injection                 | Cloud Administration Command           | Account Manipulation (6)               | Abuse Elevation Control Mechanism (6) | Abuse Elevation Control Mechanism (6) |
| Gather Victim Host Information (4)     | Acquire Infrastructure (8)           | Drive-by Compromise               | Command and Scripting Interpreter (10) | BITS Jobs                              | Access Token Manipulation (5)         | Access Token Manipulation (5)         |
| Gather Victim Identity Information (3) | Compromise Accounts (3)              | Exploit Public-Facing Application | Container Administration Command       | Boot or Logon Autostart Execution (14) | Account Manipulation (6)              | BITS Jobs                             |
| Gather Victim Network Information (6)  | Compromise Infrastructure (8)        | External                          |                                        | Boot or Logon Initialization           |                                       | Build Image on Host                   |
|                                        |                                      |                                   |                                        |                                        |                                       | Debugger Evasion                      |

An official website of the United States government [Here's how you know](#) ▾

**NSA/CSS**

About Press Room Careers History

[HOME](#) > [PRESS ROOM](#) > [NEWS & HIGHLIGHTS](#) > [ARTICLE](#)

# Kubernetes Hardening Guidance

CYBERSECURITY TECHNICAL REPORT

NEWS | March 15, 2022

**NSA, CISA release Kubernetes Hardening Guidance**

<https://attack.mitre.org/> & <https://www.nsa.gov/Press-Room/News-Highlights/Article/Article/2716980/nsa-cisa-release-kubernetes-hardening-guidance/>

# Docker Security: Vulnerability Scanner

- Eine ebenfalls sehr wichtige ergänzende Massnahme kann sicher die kontinuierliche Überwachung bereits existierender Images auf potenzielle Schwachstellen sein.
- Aktuelle Open Source Tools sind u.a.
  - ◇ [Trivy](#) – Ist ein einfacher und schneller Scanner
  - ◇ [Kubescape](#) ist ein leistungsstarkes Open-Source-Tool zur Sicherheitsbewertung und Compliance-Überprüfung von Kubernetes-Clustern. Mit der Einführung von Kubescape 3.0 wurde die Funktion zur Image-Überprüfung hinzugefügt.
- Siehe auch: [Harbor](#) und [12 Container image scanning best practices to adopt in production](#)

# Übung 05-2: Vulnerability Scanner

- Klickt auf den Link unten und spielt die Übung durch:

## ▼ Übung: Vulnerability Scanner

Container-basierte Umgebungen bestehen aus vielen Komponenten, darunter das Basis-Image, installierte Bibliotheken und Anwendungsabhängigkeiten. Jede dieser Komponenten kann potenzielle Schwachstellen enthalten, die von Angreifern ausgenutzt werden könnten. Ein Vulnerability Scanner durchsucht Container-Images nach bekannten Sicherheitslücken in den verwendeten Softwarepaketen und Konfigurationen und bietet Berichte und Empfehlungen zur Behebung dieser Schwachstellen.

### Trivy

Ist ein einfacher und schneller Scanner, der Open-Source- und kommerzielle Container-Images auf Schwachstellen überprüft.

Als erstes Überprüfen wir ein Container Image mit fehlendem `USER` Eintrag im [Dockerfile](#)

```
] : ! trivy image --scanners misconfig registry.gitlab.com/ch-mc-b/autoshop/shop:2.0.0
```

Und dann ein Container Image mit `USER` Eintrag

```
] : ! trivy image --scanners misconfig registry.gitlab.com/ch-mc-b/autoshop-ms/app/shop/shop:2.0.0
```

```
] :
```

<http://dukmaster-10-default.mshome.net:32188/notebooks/work/05-2-Scanner.ipynb>

# Weitere

- **Docker-Berechtigungen == Root-Berechtigungen** (siehe auch [A Look at Rootless Containers](#))
  - ◇ `docker run -v /:/homeroot -it ubuntu bash`
  - ◇ `cat /homeroot/etc/sudoers` # Funktioniert inkl. Änderung der System-Datei = Security Bug
  - ◇ VS
  - ◇ `podman run -v /:/homeroot -it ubuntu bash`
  - ◇ `cat /homeroot/etc/sudoers` # kein Zugriff!
- **Empfehlungen:**
  - ◇ Einen User und Gruppe im `Dockerfile` setzen
  - ◇ `setuid/setgid`-Binaries entfernen (= neuste Base Images verwenden)
  - ◇ *Neustarts begrenzen (z.B. `--restart=on-failure:10`, statt `--restart=always`) -> K8s verwenden!*
  - ◇ **In reinen Container Umgebungen (Layer 4): betreibt Docker im [Rootless](#) Modus oder verwendet [Podman](#)!**
- **Links**
  - ◇ Watch Your Containers: [Doki Infecting Docker Servers in the Cloud](#)



# Übung 05-3: Rootless Containers

- Klickt auf den Link unten und spielt die Übung durch:

## ▼ Übung: Rootless Container

### Was bedeutet "Rootless"?

"Rootless" Docker bezieht sich auf das Ausführen von Docker-Containern ohne Root-Privilegien auf dem Host-System. Standardmäßig laufen Docker-Container als Root-Benutzer, was bedeutet, dass sie vollständigen Zugriff auf das Host-Betriebssystem haben könnten, wenn es zu einem Sicherheitsvorfall kommt. Dies kann zu schwerwiegenden Sicherheitslücken führen, da ein kompromittierter Container potenziell das gesamte Host-System gefährden kann.

Sicherheitsrisiken der Ausführung als Root

- Erhöhtes Angriffspotenzial: Container, die als Root ausgeführt werden, können bei einer Kompromittierung des Containers oder einer Schwachstelle in der Container-Software den Angreifern weitreichenden Zugriff auf das Host-System gewähren.
- Seitliche Bewegungen: Ein kompromittierter Root-Container könnte möglicherweise auf andere Container und Ressourcen im gleichen Host zugreifen, was das Risiko für die gesamte Container-Infrastruktur erhöht.
- Host-Exploit: Exploits, die Root-Zugriff benötigen, können direkt auf dem Host-Betriebssystem ausgeführt werden, was die Angriffsfläche erheblich vergrößert.

Die Datei `/etc/sudoers` steuert die Berechtigungen und Regeln, die festlegen, welche Benutzer und Gruppen auf einem Linux-System das sudo-Kommando (Erlangen von Administratoren-Rechten) verwenden dürfen und welche Befehle sie mit erhöhten Rechten ausführen können.

Zu Demonstrationszwecken wollen wir versuchen die `/etc/sudoers` zu komprimieren.

Dazu mounten wir das Filesystem `/` der VMs als Verzeichnis `/homeroom` in einem Container. Ein korrekt ausgeführter Container darf diese Datei nicht anzeigen können.

Zuerst mit Docker

```
[ ]: ! docker run --rm -v /:/homeroom registry.gitlab.com/ch-mc-b/autoshop/shop:2.0.0 cat /homeroom/etc/sudoers
```

<http://dukmaster-10-default.mshome.net:32188/notebooks/work/05-3-rootless.ipynb>



# Reflexion

- Um Docker sicher einzusetzen, müssen Ihnen die potenziellen Sicherheitslücken bewusst sein, und Sie sollten die wichtigsten Tools und Techniken kennen, mit denen Sie containerbasierte Systeme absichern können.



# Lernzielkontrolle

- Sie haben einen Überblick welche Sicherheits-Aspekte bei Containern zu beachten sind.